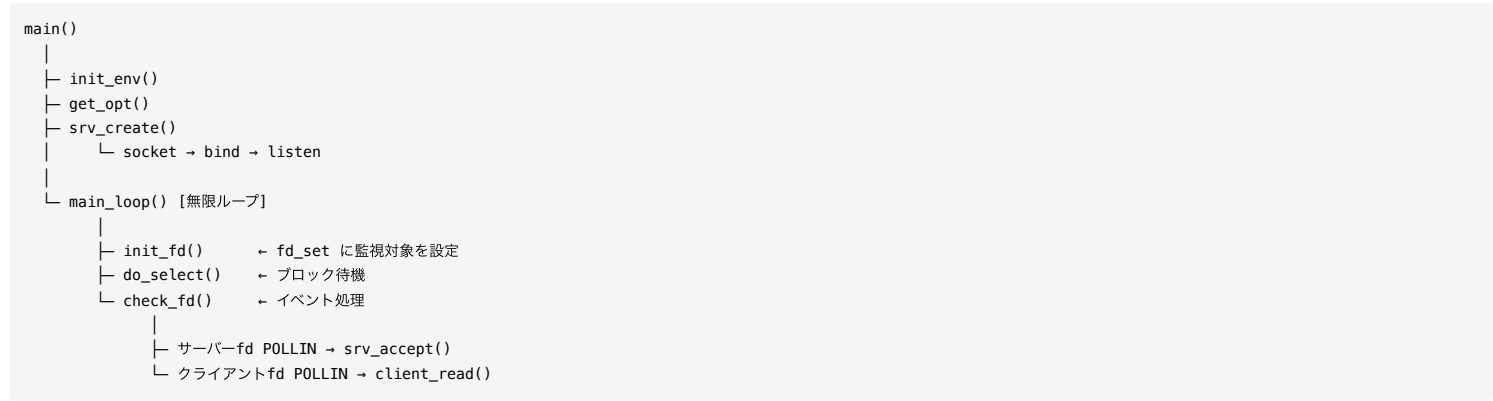




1.3 フロー図作成 (30分)

以下を自分で図示する:



チェックリスト

- ☐ `socket(PF_INET, SOCK_STREAM, ...)` が何を返すか説明できる
- ☐ `bind()` が何をするか説明できる
- ☐ `listen()` の第2引数 (backlog) の意味を説明できる
- ☐ `accept()` が何を返すか説明できる
- ☐ `FD_ZERO`, `FD_SET`, `FD_ISSET` の役割を説明できる
- ☐ `select()` の引数と戻り値を説明できる

補助リソース

- 書籍「TCP/IPソケットプログラミング C言語編」2章
- `man 2 socket`, `man 2 bind`, `man 2 listen`, `man 2 accept`
- `man 2 select`

Phase 2: select → poll 変換 (2-3時間)

目標

- `select()` と `poll()` の違いを説明できる
- `bircd` を `poll()` 版に書き換えられる

2.1 select vs poll 比較 (30分)

| 項目     | select()                     | poll()                        |
|--------|------------------------------|-------------------------------|
| fd上限   | FD_SETSIZE (1024)            | なし                            |
| 監視対象   | <code>fd_set</code> (ビットマスク) | <code>struct pollfd</code> 配列 |
| イベント指定 | 読み/書き別の <code>fd_set</code>  | <code>events</code> フィールド     |
| 結果取得   | <code>FD_ISSET()</code>      | <code>revents</code> フィールド    |

変換表:

```
// select
fd_set fd_read;
FD_ZERO(&fd_read);
FD_SET(fd, &fd_read);
select(max + 1, &fd_read, NULL, NULL, NULL);
if (FD_ISSET(fd, &fd_read)) { ... }
```

```
// poll
struct pollfd fds[MAX];
fds[i].fd = fd;
fds[i].events = POLLIN;
poll(fds, nfds, -1);
if (fds[i].revents & POLLIN) { ... }
```

2.2 poll() のイベントフラグ（15分）

| フラグ      | events | revents | 意味     |
|----------|--------|---------|--------|
| POLLIN   | ✓      | ✓       | 読み取り可能 |
| POLLOUT  | ✓      | ✓       | 書き込み可能 |
| POLLERR  | -      | ✓       | エラー発生  |
| POLLHUP  | -      | ✓       | 切断     |
| POLLNVAL | -      | ✓       | 無効なfd  |

2.3 演習: bircd を poll 版に書き換え（1.5-2時間）

変更が必要な箇所:

| ファイル        | 変更内容                       |
|-------------|----------------------------|
| bircd.h     | fd_set → struct pollfd 配列  |
| init_fd.c   | FD_ZERO/SET → pollfd配列の初期化 |
| do_select.c | select() → poll()          |
| check_fd.c  | FD_ISSET → revents チェック    |

ヒント:

```
// bircd.h に追加
#include <poll.h>
#define MAX_FDS 1024

typedef struct s_env {
    t_fd      *fds;
    struct pollfd pollfds[MAX_FDS]; // 追加
    int        nfds;                // 追加
    int        port;
    int        maxfd;
    // fd_set は削除
} t_env;
```

チェックリスト

- ☐ struct pollfd の3フィールド（fd, events, revents）を説明できる
- ☐ POLLIN , POLLOUT , POLLERR , POLLHUP の違いを説明できる
- ☐ bircd の poll 版が動作する

補助リソース

- man 2 poll
- 書籍「TCP/IPソケットプログラミング」5章（書籍には select のみだが、概念は同じ）

Phase 3: バッファリング追加（3-4時間）★最難関

目標

- TCP がバイトストリームであることを体験的に理解する
- 受信バッファで \r\n 切り出しを実装できる
- 送信バッファで非同期送信を実装できる

3.1 TCP ストリーム特性の体験（30分）

問題を体験する実験:

```
# サーバー側 (bircd を起動)
./bircd 6667

# クライアント側 (大量データ送信)
yes "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA" | head -1000 | nc localhost 6667
```

別クライアントで受信すると、recv() が複数回に分割される様子が観察できる。

**重要な理解:**

```
送信: "PRIVMSG #ch :Hello\r\nPRIVMSG #ch :World\r\n"
      ↓ TCP (バイトストリーム)
recv 1回目: "PRIVMSG #ch :Hel"
recv 2回目: "lo\r\nPRIVMSG #ch"
recv 3回目: " :World\r\n"
```

→ `recv()` の1回 ≠ メッセージの1つ

**3.2 受信バッファリング設計 (1時間)****必要なデータ構造:**

```
typedef struct s_fd {
    int    type;
    void    (*fct_read)();
    void    (*fct_write)();
    char    buf_read[BUF_SIZE + 1]; // ← これを累積バッファとして使う
    char    buf_write[BUF_SIZE + 1];
    int     buf_read_len;           // 追加: 現在のバッファ長
} t_fd;
```

**処理フロー:**

```
recv() でデータ受信
|
▼
buf_read に追記 (累積)
|
▼
buf_read 内に "\r\n" があるか?
|
├ Yes → "\r\n" までを1メッセージとして切り出し
|         残りは buf_read に残す
|         メッセージを処理
|         (複数あれば繰り返し)
|
└ No → 次の recv() を待つ
```

**実装例:**

```
void client_read(t_env *e, int cs)
{
    char    tmp[BUF_SIZE];
    int     r;
    char    *pos;

    r = recv(cs, tmp, BUF_SIZE - e->fds[cs].buf_read_len - 1, 0);
    if (r <= 0) {
        close(cs);
        clean_fd(&e->fds[cs]);
        return;
    }

    // 受信データを累積
    strcat(e->fds[cs].buf_read, tmp, r);
    e->fds[cs].buf_read_len += r;

    // \r\n を探して切り出し
    while ((pos = strstr(e->fds[cs].buf_read, "\r\n")) != NULL) {
        *pos = '\0'; // 一旦終端

        // ここで1メッセージを処理 (今はブロードキャスト)
        broadcast_message(e, cs, e->fds[cs].buf_read);

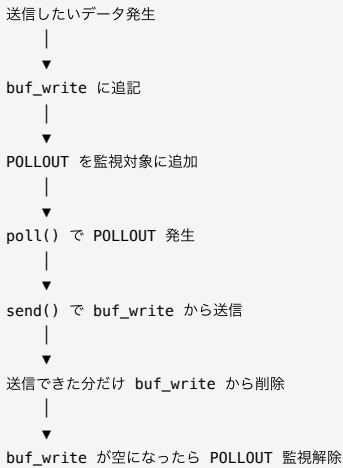
        // 処理済み部分を削除
        int msg_len = pos - e->fds[cs].buf_read + 2; // +2 for \r\n
        memmove(e->fds[cs].buf_read, pos + 2,
                e->fds[cs].buf_read_len - msg_len + 1);
        e->fds[cs].buf_read_len -= msg_len;
    }
}
```

### 3.3 送信バッファリング設計（1時間）

なぜ必要か:

- `send()` は全データを送信できるとは限らない（部分送信）
- 送信先が詰まっていると `EAGAIN` が返る

処理フロー:



実装例:

```

// メッセージをキューに追加
void queue_message(t_env *e, int cs, const char *msg)
{
    int len = strlen(msg);
    if (e->fds[cs].buf_write_len + len >= BUF_SIZE)
        return; // バッファ溢れ防止

    strcat(e->fds[cs].buf_write, msg);
    e->fds[cs].buf_write_len += len;
    // → init_fd で buf_write_len > 0 なら POLLOUT を設定
}

// POLLOUT 発生時
void client_write(t_env *e, int cs)
{
    int sent = send(cs, e->fds[cs].buf_write, e->fds[cs].buf_write_len, 0);
    if (sent <= 0) {
        close(cs);
        clean_fd(&e->fds[cs]);
        return;
    }

    // 送信済み部分を削除
    memmove(e->fds[cs].buf_write,
            e->fds[cs].buf_write + sent,
            e->fds[cs].buf_write_len - sent + 1);
    e->fds[cs].buf_write_len -= sent;
}

```

### チェックリスト

- ☐ 「TCP はバイトストリーム」の意味を説明できる
- ☐ `recv()` が1回で完全なメッセージを返さない理由を説明できる
- ☐ `\r\n` でメッセージを切り出す処理を実装できる
- ☐ 部分送信が起こる理由を説明できる
- ☐ POLLOUT を動的に制御する理由を説明できる

### 補助リソース

- 書籍「TCP/IPソケットプログラミング」6章（TCP ソケットの舞台裏）
- `bircd_analysis.md` の「部分データ処理の欠如」セクション

Phase 4: ノンブロッキングI/O（1-2時間）

目標

- ノンブロッキングI/O の必要性を説明できる
- `fcntl()` でノンブロッキング設定できる
- `EAGAIN` を正しくハンドリングできる

4.1 ブロッキング vs ノンブロッキング（30分）

| 状況                         | ブロッキング   | ノンブロッキング                       |
|----------------------------|----------|--------------------------------|
| <code>recv()</code> データなし  | 待機（ブロック） | -1, <code>errno</code> =EAGAIN |
| <code>send()</code> バッファ満杯 | 待機（ブロック） | -1, <code>errno</code> =EAGAIN |
| <code>accept()</code> 接続なし | 待機（ブロック） | -1, <code>errno</code> =EAGAIN |

なぜノンブロッキングが必要か:

`poll()` は「読める可能性がある」ことを教えるだけ。  
実際に `recv()` したら 0バイトだった、という状況がありうる。  
ブロッキングだとそこで止まってしまう。

4.2 fcntl() でノンブロッキング設定（30分）

```
#include <fcntl.h>

// ソケット作成後すぐに設定
int fd = socket(AF_INET, SOCK_STREAM, 0);
fcntl(fd, F_SETFL, O_NONBLOCK);

// accept() で得た fd にも設定
int client_fd = accept(server_fd, ...);
fcntl(client_fd, F_SETFL, O_NONBLOCK);
```

4.3 EAGAIN ハンドリング（30分）

```
void client_read(t_env *e, int cs)
{
    char tmp[BUF_SIZE];
    int r = recv(cs, tmp, BUF_SIZE, 0);

    if (r < 0) {
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            // データがないだけ、エラーではない
            return;
        }
        // 本当のエラー
        close(cs);
        clean_fd(&e->fds[cs]);
        return;
    }
    if (r == 0) {
        // 正常切断
        close(cs);
        clean_fd(&e->fds[cs]);
        return;
    }
    // r > 0: データ受信成功
    // ...
}
```

チェックリスト

- ☐ ノンブロッキングI/O が必要な理由を説明できる
- ☐ `fcntl(fd, F_SETFL, O_NONBLOCK)` を書ける
- ☐ `EAGAIN` と本当のエラーを区別できる

補助リソース

- `man 2 fcntl`
- `man 2 recv` の `EAGAIN` の説明

## Phase 5: C++98化（2時間）

### 目標

- Phase 1-4 の成果物を C++98 で書き直せる

### 5.1 データ構造の変換（1時間）

| C (birdc)                        | C++98                                                                               |
|----------------------------------|-------------------------------------------------------------------------------------|
| <code>t_fd fds[MAX]</code>       | <code>std::map&lt;int, Client&gt;</code> または <code>std::vector&lt;pollfd&gt;</code> |
| <code>char buf_read[SIZE]</code> | <code>std::string recv_buffer</code>                                                |
| 関数ポインタ                           | 仮想関数 or 単純な if 分岐                                                                   |
| <code>malloc/free</code>         | <code>new/delete</code> or コンテナ                                                     |

### 5.2 クラス設計例（1時間）

```
class Server {
private:
    int _serverFd;
    int _port;
    std::vector<struct pollfd> _pollfds;
    std::map<int, std::string> _recvBuffers;
    std::map<int, std::string> _sendBuffers;

public:
    Server(int port);
    ~Server();

    void run(); // メインループ

private:
    void setupSocket();
    void acceptNewClient();
    void receiveData(int clientFd);
    void sendData(int clientFd);
    void disconnectClient(int clientFd);
    void queueResponse(int clientFd, const std::string& msg);
};
```

### チェックリスト

- ☐ `std::vector<struct pollfd>` で fd 管理できる
- ☐ `std::map<int, std::string>` でバッファ管理できる
- ☐ C++98 の範囲で実装できる（C++11機能は使わない）

## 最終チェックリスト

Phase 完了後、以下を全て説明・実装できることを確認:

### 知識

- ☐ TCP がバイトストリームである理由
- ☐ `select()` と `poll()` の違い
- ☐ ノンブロッキング I/O の必要性
- ☐ `POLLOUT` を動的に制御する理由
- ☐ 受信バッファリングの必要性
- ☐ 送信バッファリングの必要性

### 実装

- ☐ `socket/bind/listen/accept` でサーバー起動
- ☐ `poll()` で I/O 多重化
- ☐ `\r\n` でメッセージ切り出し
- ☐ 非同期送信（`POLLOUT` 制御）
- ☐ ノンブロッキング + `EAGAIN` ハンドリング
- ☐ クライアント切断検知（`POLLERR`, `POLLHUP`, `recv=0`）

補助リソースまとめ

| Phase | 必須リソース                                     |
|-------|--------------------------------------------|
| 1     | 書籍2章, man socket/bind/listen/accept/select |
| 2     | man poll                                   |
| 3     | 書籍6章, bircd_analysis.md                    |
| 4     | man fcntl, man recv                        |
| 5     | C++98 リファレンス                               |

書籍: 「TCP/IPソケットプログラミング C言語編」

- URL: <https://www.ohmsha.co.jp/book/9784274065194/>
- 特に 2章、5章、6章

関連ドキュメント

- `bircd_analysis.md` - bircd の構造分析
- `reading_guide_A.md` - A担当 (Network/IO) 向け読書ガイド
- `myIRCD/src/Server.cpp` - 完成形の参考実装